

Тема 12 – Линејни динамични АТД

Видове свързани списъци - има основно три вида:

1. Единично свързан списък
2. Двойно свързан списък
3. Кръгово свързан списък
4. Двойно кръгово свързан списък

Тема 12 – Линејни динамични АТД

Създавање и обхождане на единично свързан списък

Всеки свързан списък има: «глава» (head) – указател, който сочи към първия възел от списъка и всеки възел има указател, който сочи към следващия елемент в списъка, като указателят на последния възел е NULL, което показва край на свързания списък.

```
struct node {  
    int data;  
    struct node *next;  
};
```



Тема 12 – Линејни динамични АТД

Списък с основни операции със свързани списъци:

1. Преминаване - достъп до всеки елемент от свързания списък
2. Вмъкване - добавя нов елемент към свързания списък
3. Изтриване - премахва съществуващите елементи
4. Търсене - намиране на възел в свързания списък
5. Сортиране - сортиране на възлите от свързания списък

Тема 12 – Линејни динамични АТД

Преминавање през свързан списък

Показването на съдържанието на свързан списък е много лесно. Продължаваме да преместваме временния възел към следващия и показваме съдържанието му.

Когато `temp` е `NULL`, знаем, че сме достигнали края на свързания списък, така че излизаме от цикъла `while`.

```
struct node *temp = head;  
printf("\n\nList elements are - \n");  
while(temp != NULL) {  
    printf("%d --->", temp->data);  
    temp = temp->next;  
}
```

Тема 12 – Линејни динамични АТД

Може да добавяте елементи в началото, средата или края на свързания списък.

1. Добавяне в началото

- * Алокира се памет за нов възел
- * Записват се данните за новия възел
- * Променя се указателя на новия възел да сочи към главата на списъка
- * Променя се главата, за да сочи към новия създаден възел

```
struct node *newNode;
```

```
newNode = malloc(sizeof(struct node));
```

```
newNode->data = 4;
```

```
newNode->next = head;
```

```
head = newNode;
```

Тема 12 – Линејни динамични АТД

1. Добавяне в края

- * Алокира се памет за нов възел
- * Записват се данните за новия възел
- * Преминува се към последния възел
- * Променя се указателя на последния възел да сочи към новия възел, а указателя на новия възел ще сочи към NULL

```
struct node *newNode;
```

```
newNode = malloc(sizeof(struct node));
```

```
newNode->data = 4;
```

```
newNode->next = NULL;
```

```
struct node *temp = head;
```

```
while(temp->next != NULL){
```

```
    temp = temp->next;
```

```
}
```

```
temp->next = newNode;
```

Тема 12 – Линејни динамични АТД

1. Добавяне в средата

- Алокира се памет за нов възел
- * Записват се данните за новия възел
- * Преминува се към възел, който е точно преди желаната позиция за новия възел
- * Променя се указателя на възела да сочи към новия възел, а указателя на новия възел ще сочи към следващия възел.

```
struct node *newNode;
```

```
newNode = malloc(sizeof(struct node));
```

```
newNode->data = 4;
```

```
struct node *temp = head;
```

```
for(int i=2; i < position; i++) {
```

```
    if(temp->next != NULL) {
```

```
        temp = temp->next;
```

```
    }
```

```
}
```

```
newNode->next = temp->next;
```

```
temp->next = newNode;
```

Тема 12 – Линејни динамични АД

Изтриване в началото на списъка

- * Указателят към списъка (главата) се насочва към втория възел

```
head = head->next;
```

Изтриване в края на списъка

- * Позиционираме се на предпоследния елемент
- * Насочваме неговия указател към NULL

```
struct node* temp = head;
```

```
while(temp->next->next!=NULL){
```

```
    temp = temp->next;
```

```
}
```

```
temp->next = NULL;
```


Тема 12 – Линејни динамични АТД

Изтриване в средата на списъка

- * Позиционираме се на възела, който е преди възела за изтриване
- * Насочваме неговия указател към възела, към който сочи указателя на елемента за изтриване

```
head = head->next;
```

Изтриване в края на списъка

- * Позиционираме се на предпоследния елемент
- * Насочваме неговия указател към NULL

```
for(int i=2; i< position; i++) {  
    if(temp->next!=NULL) {  
        temp = temp->next;  
    }  
}  
temp->next = temp->next->next;
```

Тема 12 – Линејни динамични АТД

Търсене на елемент в списъка

// Search a node, key – търсената стойност

```
bool searchNode(struct Node** head_ref, int key) {
```

```
    struct Node* current = *head_ref;
```

```
    while (current != NULL) {
```

```
        if (current->data == key) return true;
```

```
        current = current->next;
```

```
    }
```

```
    return false;
```

```
}
```

Тема 12 – Линејни динамични АД

Сортирање на елементите в списъка

// Sort the linked list – Метод на мехурчето

```
void sortLinkedList(struct Node** head_ref) {  
    struct Node *current = *head_ref, *index = NULL;  
    int temp;  
    if (head_ref == NULL) {  
        return;  
    } else {  
        while (current != NULL) {  
            // index points to the node next to current  
            index = current->next;  
            while (index != NULL) {  
                if (current->data > index->data) {  
                    temp = current->data;  
                    current->data = index->data;  
                    index->data = temp;  
                }  
                index = index->next;  
            }  
            current = current->next;  
        }  
    }  
}
```

Тема 12 – Линејни динамични АТД

Двойно свързан списък

Двойно свързан списък е двупосочен свързан списък. Така че можете да го обхождате и в двете посоки.

За разлика от единично свързаните списъци, неговите възли съдържат един допълнителен указател, наречен предишен указател. Този указател сочи към предишния възел. Това го прави идеална структура от данни за приложения, които изискват възможност за придвижване напред и назад през списък с данни.



Тема 12 – Лінійні динамічні АТД

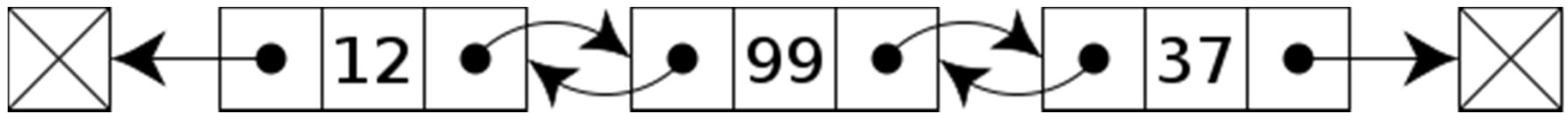
```
struct node {
```

```
int data;
```

```
struct node *next;
```

```
struct node *prev;
```

```
}
```

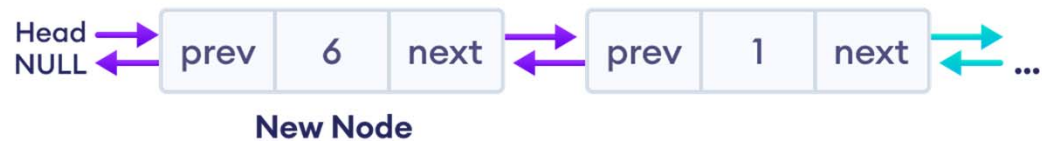
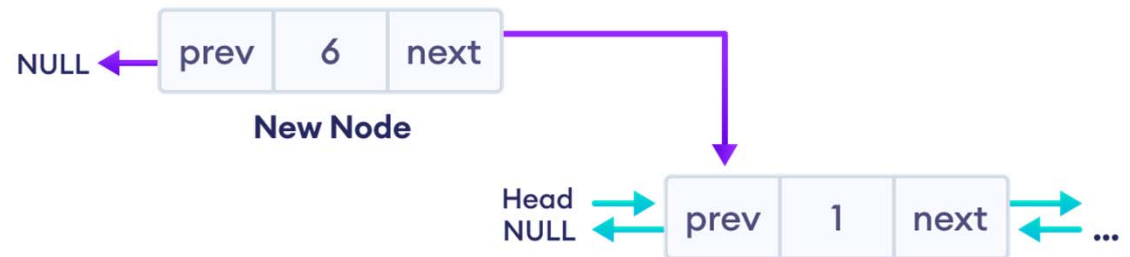


Тема 12 – Линејни динамични АД

Добавяне на възел в началото на двойно свързан списък



New Node



Тема 12 – Линејни динамични АТД

Добавяне на възел в началото на двойно свързан списък

// insert node at the front

```
void insertFront(struct Node** head, int data) {
```

```
    // allocate memory for newNode
```

```
    struct Node* newNode = new Node;
```

```
    // assign data to newNode
```

```
    newNode->data = data;
```

```
    // point next of newNode to the first node of the doubly linked list
```

```
    newNode->next = (*head);
```

```
    // point prev to NULL
```

```
    newNode->prev = NULL;
```

```
    // point previous of the first node (now first node is the second node) to newNode
```

```
    if ((*head) != NULL)
```

```
        (*head)->prev = newNode;
```

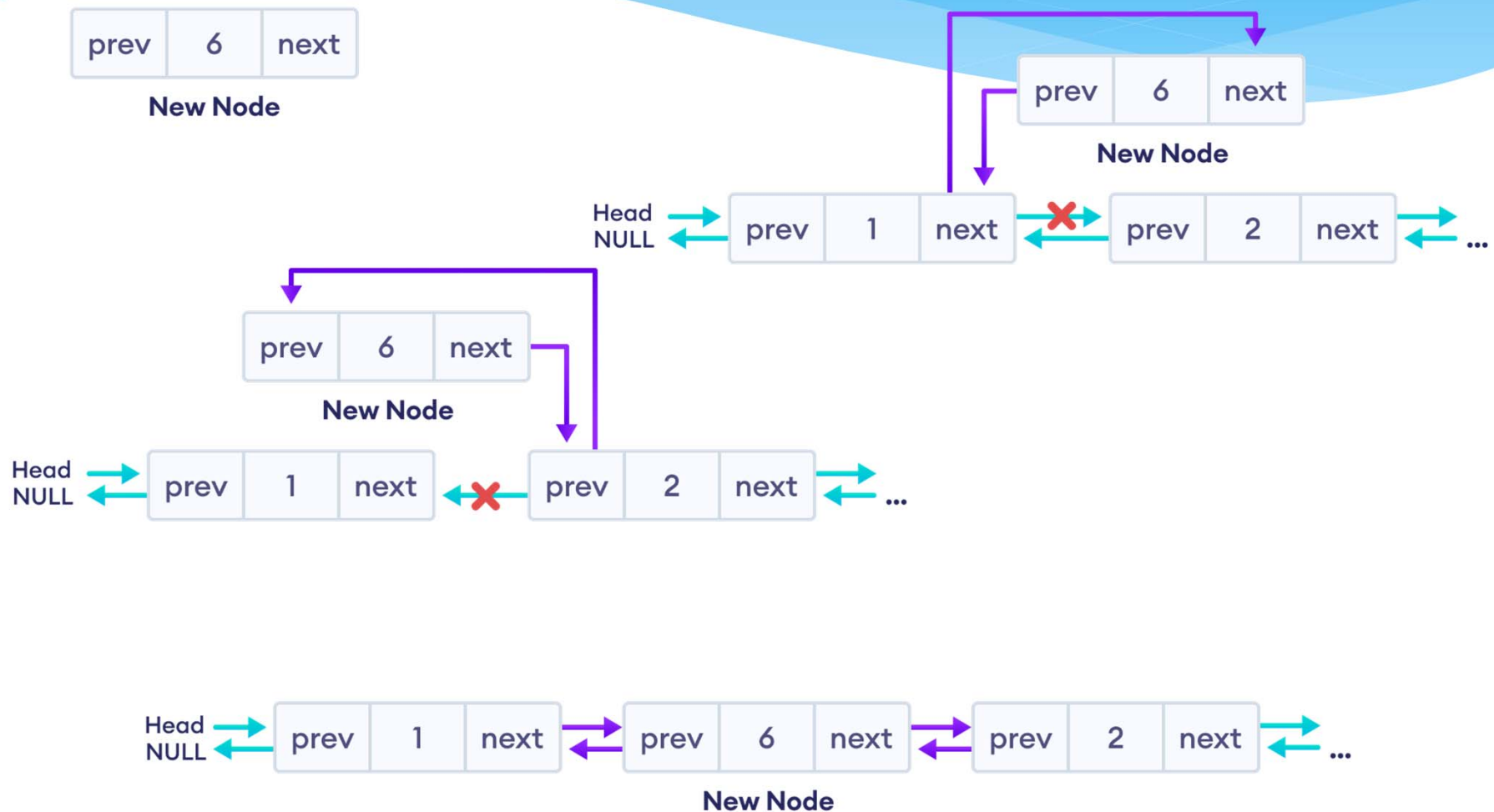
```
    // head points to newNode
```

```
    (*head) = newNode;
```

```
}
```

Тема 12 – Линејни динамични АД

Добавяне на възел в средата на двойно свързан списък



Тема 12 – Линејни динамични АТД

Добавяне на възел в средата на двойно свързан списък

// insert a node after a specific node

```
void insertAfter(struct Node* prev_node, int data) {
```

```
    // check if previous node is NULL
```

```
    if (prev_node == NULL) {
```

```
        cout << "previous node cannot be NULL";
```

```
        return;
```

```
    }
```

```
    // allocate memory for newNode
```

```
    struct Node* newNode = new Node;
```

```
    // assign data to newNode
```

```
    newNode->data = data;
```

```
    // set next of newNode to next of prev node
```

```
    newNode->next = prev_node->next;
```

```
    // set next of prev node to newNode
```

```
    prev_node->next = newNode;
```

```
    // set prev of newNode to the previous node
```

```
    newNode->prev = prev_node;
```

```
    // set prev of newNode's next to newNode
```

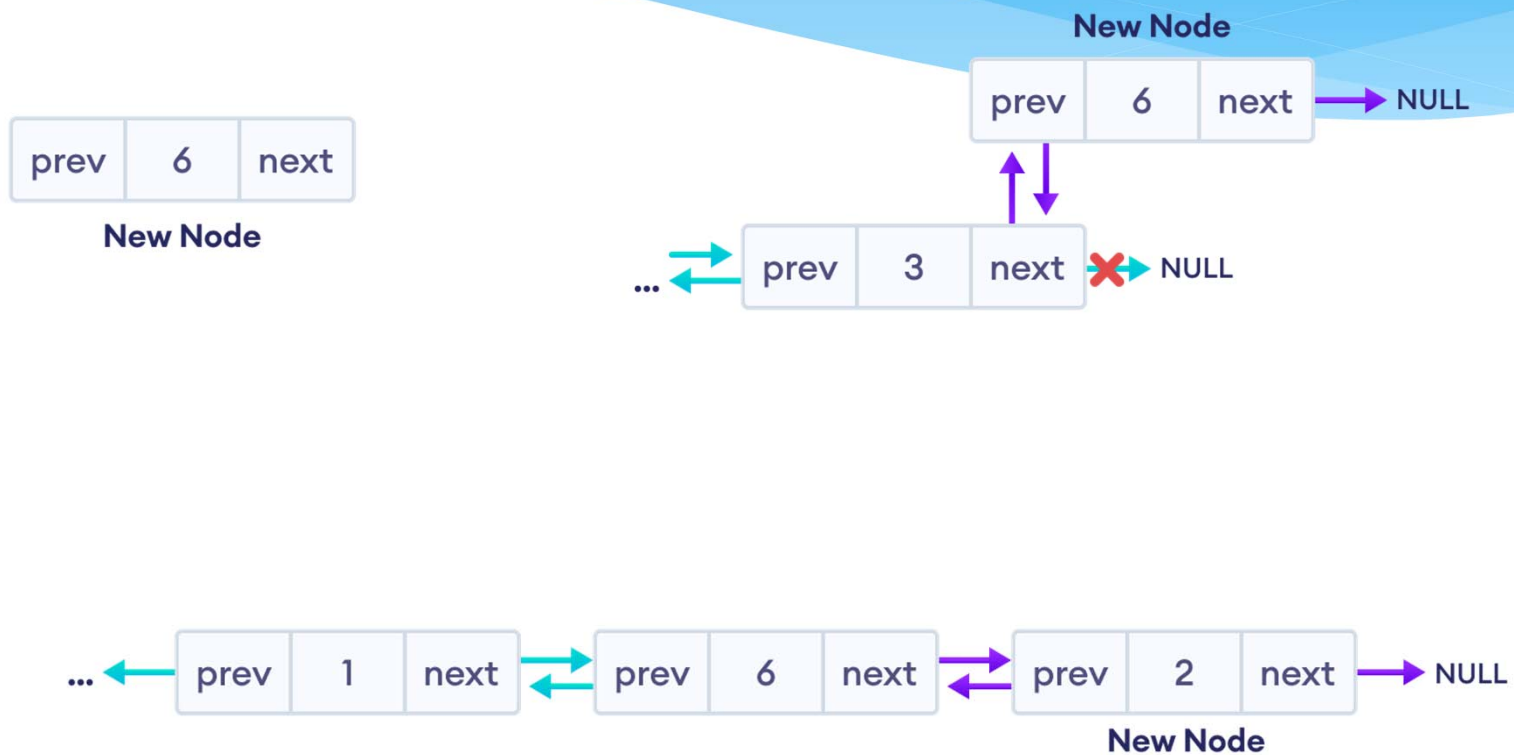
```
    if (newNode->next != NULL)
```

```
        newNode->next->prev = newNode;
```

```
}
```

Тема 12 – Линејни динамични АТД

Добавяне на възел в края на двойно свързан списък



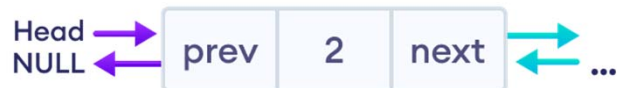
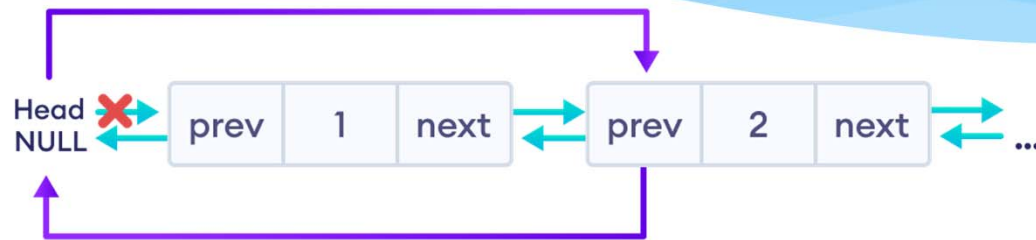
Тема 12 – Линејни динамични АТД

Добавяне на възел в края на двойно свързан списък

```
void insertEnd(struct Node** head, int data) {  
    // allocate memory for node  
    struct Node* newNode = new Node;  
    // assign data to newNode  
    newNode->data = data;  
    // assign NULL to next of newNode  
    newNode->next = NULL;  
    // store the head node temporarily (for later use)  
    struct Node* temp = *head;  
    // if the linked list is empty, make the newNode as head node  
    if (*head == NULL) {  
        newNode->prev = NULL;  
        *head = newNode;  
        return; }  
    // if the linked list is not empty, traverse to the end of the linked list  
    while (temp->next != NULL)  
        temp = temp->next;  
    // now, the last node of the linked list is temp // point the next of the last node (temp) to newNode.  
    temp->next = newNode;  
    // assign prev of newNode to temp  
    newNode->prev = temp; }
```

Тема 12 – Линејни динамични АД

Изтриване на възел в началото на двойно свързан списък



Free the space of the first node

Тема 12 – Линејни динамични АТД

Изтриване на възел в началото на двойно свързан списък

```
if (*head == del_node)
```

```
    *head = del_node->next;
```

```
if (del_node->prev != NULL)
```

```
    del_node->prev->next = del_node->next;
```

```
free(del);
```

Тема 12 – Линејни динамични АТД

Изтриване на възел в средата на двойно свързан списък



Тема 12 – Линејни динамични АТД

Изтриване на възел в средата на двойно свързан списък

```
if (del_node->next != NULL)
```

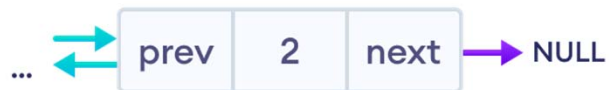
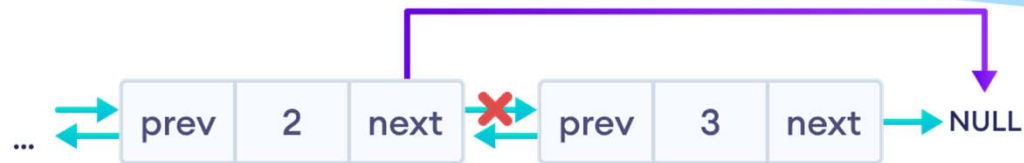
```
    del_node->next->prev = del_node->prev;
```

```
if (del_node->prev != NULL)
```

```
    del_node->prev->next = del_node->next;
```

Тема 12 – Линейни динамични АТД

Изтриване на възел в края на двойно свързан списък



Тема 12 – Линејни динамични АТД

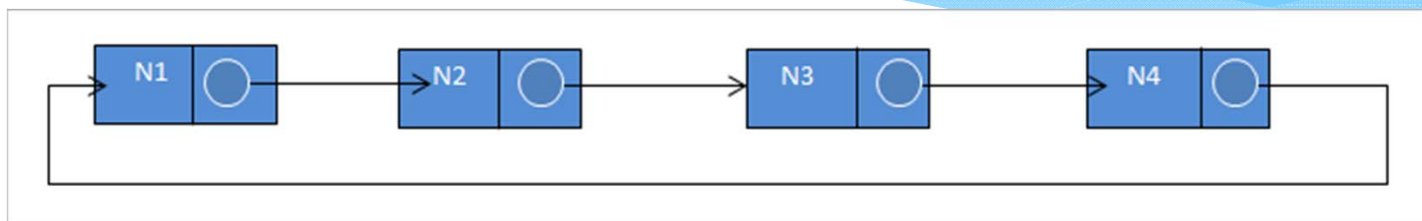
Изтриване на възел в края на двойно свързан списък

```
if (del_node->prev != NULL)
```

```
    del_node->prev->next = del_node->next;
```

Тема 12 – Линейни динамични АТД

Кръгово свързан списък



Двойно кръгово свързан списък

